Universidad Nacional Autónoma de México — Facultad de Ingeniería
Fundamentos de Sistemas Embebidos — Lab 5: Multimedia Kiosk
Rafael Ortega de la Paz | 420054085

1. Introduction

This lab is about configuring a Raspberry Pi as a multimedia kiosk, a dedicated device that plays audio and video without anyone operating it. In my own words, the goal was to take a Raspberry Pi 4 Model B running Raspberry Pi OS Lite and make it boot straight into a Python program that plays a video while controlling its volume, then loops a slideshow of images and switches to the images of a USB drive as soon as one is inserted, with six push buttons on the GPIO pins to control playback. For this I used FFmpeg and VLC for playback, python-vlc to control the player from code, pyudev to detect the drive and gpiozero to read the buttons.

2. Background

Wolf (2016) defines an embedded computing system as a device that contains a programmable computer but is not itself meant to be a general-purpose computer. A kiosk fits that definition: a low-cost processor and a few peripherals dedicated to one task behind a minimal interface. Hallinan (2010) treats the start-up sequence of an embedded Linux system as something the designer customizes so that the device comes up running its application instead of waiting at a login prompt, which is what the kiosk mode of this lab does through `/etc/rc.local`.

Playback is split between two tools. The `ffplay` utility from FFmpeg is a minimal player useful for quick tests, while VLC can also be driven from code through its libVLC bindings. The lab manual (Matamoros, 2025) uses python-vlc for this: a player object receives a media object and starts playback asynchronously, so the program decides how long it runs.

Detecting the USB drive relies on the Linux device model. As Corbet et al. (2005) explain, the kernel notifies user space when devices are added or removed, and udev reacts to those events. The pyudev library wraps libudev so that a Python program can wait for them and filter them, here to block devices of type partition. Once the partition appears it still has to be mounted, that is, attached to the directory tree before its files can be read (Silberschatz et al., 2018), which the script requests through `udisksctl`.

3. Development

The lab was done on a Raspberry Pi 4 Model B with Raspberry Pi OS Lite (Debian 13, kernel 6.18). I confirmed it with `cat /etc/os-release` and `systemctl get-default`, which returned `multi-user.target`, so there is no graphical session. I installed the packages of the manual (nodm, ffmpeg, pulseaudio, vlc, python3-vlc, python3-pyudev, wget) and two that it does not list: git, and udisks2 because `usbdetect.py` calls `udisksctl`. The test media was downloaded with `wget` and extracted to `/home/pi`, giving `video.mp4` (464x352, H.264 and AAC, 212 s) and four JPEG images. The audio output was set to the 3.5 mm jack with `raspi-config`.

`ffplay` played the video with sound. For VLC I built an M3U playlist with `realpath` and ran `vlc --loop playlist.m3u` over SSH. The images appeared on the monitor even though there is no X server: the log showed the xcb errors and then `drm_vout` opening, so VLC fell back to drawing through DRM, and it also decoded the video with the DRM hardware accelerator. The PulseAudio connection errors did not stop playback.

Experiment 1 (`exp1.py`). The script plays `video.mp4`, sleeps 10 s and calls `stop()`,

then loops forever over the sorted images, showing each for 3 s. I did not use the `while player.is_playing` loop of the example: `is_playing` is a method, so without parentheses the condition is always true.

Experiment 2 (`exp2.py`). I merged `usbdetect.py` into the slideshow. A pyudev monitor filtered to subsystem `block` and device type `partition` is polled with a timeout equal to the remaining slide time, which replaces the plain sleep so the loop notices an `add` event without blocking. On insertion the script mounts the partition with `udisksctl`, gets the mount point with `findmnt`, lists the `.jpg` and `.png` files in its root and replaces the slideshow list with them. Testing with copies of the sample images could not show the switch, so I made the script print each file shown and used four different pictures on a FAT32 drive. The output went from `/home/pi/pictures/pic01.jpg` to `Mounted /dev/sda1 at /media/root/FSEM` and then to `/media/root/FSEM/foto01.jpg` through `foto04.jpg`, in a loop. The drive must be inserted after the script starts and the images must be in its root; the `._` files that macOS creates on FAT had to be removed with `dot_clean`.

Experiment 3 (`exp3.py`). The video plays for 20 s while a loop calls `audio_set_volume` every 0.1 s: t/5 $\times$ 100 % during the first 5 s, 100 % until second 15, and (20 $-$ t)/5 $\times$ 100 % in the last 5 s. The console printed the volume each second: 0 % at t = 0, 20 % at t = 1, 100 % at t = 5, 99 % at t = 15 and 19 % at t = 19, after which the slideshow started. The volume change was clearly audible.

Experiment 4 (`exp4.py`). Six push buttons on a breadboard connect GPIO2, 3, 4, 17, 27 and 22 (header pins 3, 5, 7, 11, 13 and 15) to ground (pin 9) and act as previous, next, stop, pause/resume, volume up and volume down. With the internal pull-ups enabled through gpiozero (`Button(pin, pull_up=True)`) no resistors are needed, and a pressed button reads low. A small script that printed the name of each button confirmed the wiring before I touched the kiosk. In the final script the button callbacks only store a command and the main loop, which already handled the slide timer and the udev poll, executes it, so the player is never called from two threads. The playlist is a Python list with a single `MediaPlayer` rather than a `MediaListPlayer`, because the volume fade and the USB switch already needed direct control of the player. Pause freezes the slide and fade clocks, pause after stop plays the current item again, and the volume buttons change a master level that the fade multiplies and repeat while held. The video plays only once: afterwards the list is reduced to the slides, since my first version wrapped around to the video again. The console log showed every button working on both the video and the slides.

Kiosk mode. The final script (`exp4.py`) was copied to `/home/pi/minikiosk.py`. In this release `/etc/rc.local` does not exist, so I created it with the line `python3 /home/pi/minikiosk.py &` before `exit 0` and made it executable. After `sudo reboot` the Pi played the video and then the slideshow with no login, and inserting the USB drive switched the images. Since local login is no longer possible in this mode, the remaining work was done over SSH.

Video. The demonstration of the kiosk, from power-on to the USB switch and the six buttons, is available at `https://www.youtube.com/watch?v=Syh4dkxWpxA`.

4. Questionnaire

1. Volume. `player.audio_set_volume(n)` sets the volume from 0 to 100 and `player.audio_get_volume()` reads it, for example `player.audio_set_volume(player.audio_get_`

`volume() + 10`; `audio_toggle_mute()` mutes and unmutes.

2. Full screen. `player.set_fullscreen(True)` turns it on and `player.toggle_fullscreen()` switches it; the `--fullscreen` option can also be passed with `vlc.Instance('--fullscreen')`.

3. Playlist control. A `vlc.MediaList` is played with a `vlc.MediaListPlayer`: `lp = vlc.MediaListPlayer(); lp.set_media_list(vlc.MediaList(paths)); lp.play()`. Then `lp.next()` and `lp.previous()` move through the list, `lp.pause()` pauses (and `lp.set_pause(0)` resumes), `lp.stop()` stops it and `lp.play_item_at_index(i)` jumps to an item.

5. Conclusions

A Raspberry Pi can work as an unattended kiosk with very little software: a minimal OS, VLC controlled through python-vlc, pyudev for the USB drive and one line in `rc.local`. Experiments 1 to 4 behaved as specified. The main design point was keeping timing and media control inside the script: using a udev poll with a timeout instead of a sleep let the same loop handle both the 3 s slides and the insertion event.

The difficulties were mostly in the setup. The example code needed fixes (`is_playing` without parentheses, and `miniusbdetect.py` comparing the action with `'partition'` instead of `'add'`); git and udisks2 were missing from the package list; `rc.local` had to be created; and the first USB test was inconclusive because the drive held copies of the sample images. Since kiosk mode removes local login, it should be the last step of the lab.

References

Corbet, J., Rubini, A., and Kroah-Hartman, G. (2005). *Linux Device Drivers*, 3rd ed. O'Reilly Media.

Hallinan, C. (2010). *Embedded Linux Primer: A Practical Real-World Approach*, 2nd ed. Prentice Hall.

Matamoros de Maria y Campos, J. M. (2025). *Práctica 5: Kiosco multimedia*. Fundamentos de Sistemas Embebidos, UNAM.

Silberschatz, A., Galvin, P. B., and Gagne, G. (2018). *Operating System Concepts*, 10th ed. Wiley.

Wolf, M. (2016). *Computers as Components: Principles of Embedded Computing System Design*, 4th ed. Morgan Kaufmann.